

Python II:

lists, slicing, comprehensions

On Friday you walked along a list. Today you take one apart — index it, slice it, copy it without meaning to, nest it into a grid, and finally hit the wall that makes NumPy necessary on Friday.

Unit 1 • Foundations

80 minutes

Lab 2 • a grid, two ways

Laptops open

Where we are Lists Slicing The copy trap Comprehensions Grids The wall Lab 2 Before Friday

Where we are

Friday gave you four things: a value, a name, a function, and a loop with an accumulator. That was enough to write `average(values)`, and `average` is genuinely the shape of everything in unit 2.

But look at what you were handed rather than what you wrote. The *list* arrived from nowhere. You walked along it and never asked what it was, how to reach into it, or what happens when you copy one. Today is that, and it matters more than it sounds: by session 8 a list of numbers becomes a layer of a network, and by session 23 a list of lists is an image.



Lab 1, briefly

Two things came up in most rooms. The negative case in `bigger` — anyone who started from zero got it wrong, on purpose, and that is bug two from the diagnostic. And several people put `return` inside the loop in `total`, which is bug three. If both of those are now obvious to you, Friday did its job.

PART 1 • 12 MINUTES

A list, and how to reach into it

Several values under one name, in a definite order, reachable by position.

1

the basics – run every line

PYTHON

```
scores = [70, 85, 60, 95]

scores[0]      # 70   the FIRST element – counting starts at zero
scores[3]      # 95   the fourth
scores[-1]     # 95   the last, without knowing how long the list is
scores[-2]     # 60   second from the end

len(scores)    # 4    how many items
scores[4]      # IndexError – there is no fifth element
```

1 2
3 4

Zero-based indexing, said once, properly

The first element is at position 0 and the last is at position `len(scores) - 1`. This is not a quirk to tolerate; it is why `-1` means "the last" and why slicing works as cleanly as it does in ten minutes' time. If you came from a language that counts from one, expect to make this mistake twice today and then stop.

Changing a list, and growing one

PYTHON

```
scores = [70, 85, 60, 95]

scores[1] = 90      # replace in place → [70, 90, 60, 95]
scores.append(55)   # add to the end → [70, 90, 60, 95, 55]
len(scores)         # 5
```

`append` is the one you will use constantly: it is how a loop builds a list of results, which is the pattern behind every plot you draw for the rest of the term.

the loop that builds a list – remember this shape

PYTHON

```
squares = []          # start empty, before the loop
for x in [1, 2, 3, 4]:
    squares.append(x * x)  # grow it, inside
print(squares)          # [1, 4, 9, 16]
```



This is Friday's accumulator again

Same three-step shape – create before the loop, update inside, use after – with a list in place of a number. Recognising that these are one pattern rather than two is worth more than memorising either.

Two more things you will need today

PYTHON

```
range(5)           # 0, 1, 2, 3, 4 – five numbers, starting at zero
range(2, 6)        # 2, 3, 4, 5 – from 2 up to but NOT including 6

for i in range(len(scores)):
    print(i, scores[i])  # position and value together
```

Two ways to walk a list, then: `for s in scores` when you want the values, and `for i in range(len(scores))` when you need the positions too. Prefer the first. You need the second exactly when you must know *where* something is — which is the grid problem in part 5, and was question B3 on the diagnostic.

"Up to but not including" is the theme of this session

`range(2, 6)` stops at 5. A slice will do the same thing in a moment. Anyone who answered 15 instead of 10 on the diagnostic's first question met this rule already: `range(1, 5)` gives 1, 2, 3, 4 and sums to 10. It is one rule, applied everywhere, and once you accept it the off-by-one errors stop.

PART 2 • 14 MINUTES

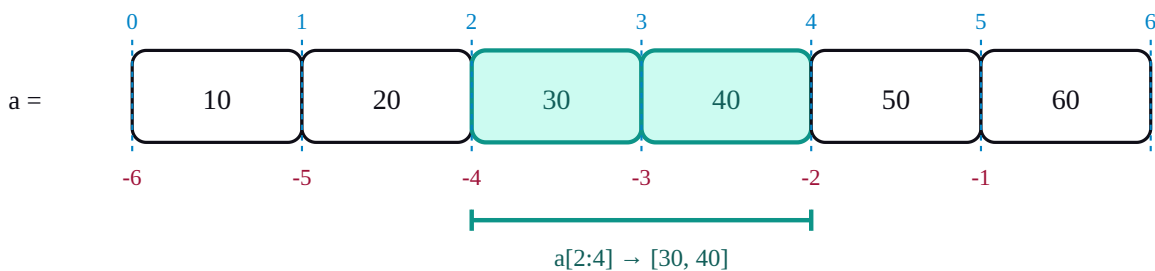
Slicing — spend real time here

2

The single highest-value ten minutes in this session. Every dataset operation for the rest of the semester is a slice.

The mental model that makes slicing stop being confusing: **indices do not label the elements, they label the gaps between them.** A slice `a[i:j]` takes everything between gap *i* and gap *j*.

INDEX THE FENCEPOSTS, NOT THE FENCE PANELS



Read `a[2:4]` as "from gap 2 to gap 4". Two gaps apart, so two elements. The length of a slice is always `stop - start` — no off-by-one to reason about, ever again.

run every one of these — outputs shown are real

PYTHON

```
a = [10, 20, 30, 40, 50, 60]

a[2]      # 30          one element (an int, not a list)
a[2:4]    # [30, 40]    gap 2 to gap 4
a[:3]     # [10, 20, 30] from the start
a[3:]     # [40, 50, 60] to the end
a[-1]     # 60         last element
a[-2:]    # [50, 60]   last two
a[::-2]    # [10, 30, 50] every second
a[::-1]    # [60, 50, 40, 30, 20, 10] reversed
a[:]      # a full copy — remember this one, Part 5 needs it
```



Why this is worth ten minutes of an eighty-minute session

In session 16 you will write `X_train = X[:800]` and `X_test = X[800:]` to split a dataset. In session 16 you will write `X[i:i+32]` to take a mini-batch. In NumPy on Friday the same syntax gains a second dimension: `X[0:5, 2:4]` — rows 0 to 5, columns 2 to 4. Nothing new to learn on Friday if you own this today.

PART 3 • 8 MINUTES

The copy that is not a copy

3

This one catches experienced programmers too, and it causes some of the nastiest bugs you will meet in NumPy later. Meet it now, on a three-element list, where it is still funny rather than expensive.

predict the output before you read it

PYTHON

```
a = [1, 2, 3]
b = a          # NOT a copy – b is a second label on the same list
b[0] = 99

print(a)       # [99, 2, 3] ← a changed, and you never touched a
```

The fix is the slice you met a moment ago:

an actual copy

PYTHON

```
a = [1, 2, 3]
c = a[:]      # or list(a) – a genuine copy
c[0] = 7

print(a, c)   # [1, 2, 3] [7, 2, 3]
```



Remember this one — it comes back much worse

NumPy slices behave the same way, and for arrays the consequences are larger: a slice of an array is usually a view onto the original memory, so writing into the slice modifies the original array. When a training run in session 16 produces results that make no sense, this is the second thing to check. The first is shapes.

PART 4 • 10 MINUTES

Comprehensions — "do this to every element"

4

One line replaces four, and more importantly it trains the habit of thinking about a whole collection at once.

the same job, twice

PYTHON

```
# the loop you already know
squares = []
for x in [1, 2, 3, 4]:
    squares.append(x**2)

# the same thing, as one expression
squares = [x**2 for x in [1, 2, 3, 4]]
```

Read it right to left: *for every x in this list, give me x squared*. You can filter in the same breath:

with a condition, and over a range

PYTHON

```
evens = [x for x in range(10) if x % 2 == 0] # [0, 2, 4, 6, 8]

# range(a, b) counts from a up to but NOT including b — a slice, again
xs = [i/10 for i in range(-30, 31)] # -3.0, -2.9, ... 3.0
print(len(xs)) # 61
```



Why divide by 10 instead of using a decimal step?

`range` only counts in whole numbers. So we count in integers and divide — a small, ordinary workaround. On Friday `np.linspace(-3, 3, 61)` replaces the whole line, and that substitution is a fair summary of what NumPy is for.

PART 5 · 10 MINUTES

A list of lists is a grid

5

And a grid, in this course, is a photograph. That is not a metaphor — it is the same object.

Put lists inside a list and you have rows. Each inner list is one row; each position inside it is one column.

three rows, three columns

PYTHON

```
grid = [[4, 7, 2],
        [9, 1, 8],
        [3, 6, 5]]

grid[1] # [9, 1, 8] the whole second row
grid[1][0] # 9 row 1, column 0
grid[0][2] # 2 row 0, column 2

len(grid) # 3 how many rows
len(grid[0]) # 3 how many columns
```



Row first, then column. Always.

`grid[1][0]` is row 1, column 0 — *not* $x = 1, y = 0$. If you learned coordinates in mathematics, this is the reverse of what your hand wants to write. It is the convention in NumPy, in PyTorch, in every image library, and in every line of this course from here to December. Say "row, column" out loud when you write it, for about a week, and then it will be automatic.

Walking a grid needs two loops

the nested loop – the shape of diagnostic question B3

PYTHON

```
for r in range(len(grid)):      # each row
    for c in range(len(grid[0])): # each column in that row
        print(r, c, grid[r][c])
```

The outer loop picks a row and holds it still; the inner loop runs all the way across before the outer one moves on. Nine passes for a 3×3 grid, in the order (0,0), (0,1), (0,2), (1,0) and so on. Trace it on paper if that order is not obvious – the same two columns you drew on Friday, with `r` and `c` this time.

the accumulator again, now over two dimensions

PYTHON

```
def grid_total(g):
    running = 0
    for row in g:
        for value in row:
            running = running + value
    return running

print(grid_total(grid))    # 45
```

WHY THIS SMALL THING IS A LARGE THING

A grayscale photograph is exactly this: a list of rows, each row a list of numbers from 0 (black) to 255 (white). A 28×28 handwritten digit is 28 lists of 28 numbers. When we get to session 23 and open a real image, no new idea arrives – only a bigger grid and a faster container. You have just learned the data structure of computer vision, in a form small enough to check by hand.



Do this now – three minutes

Write `row_totals(g)`, returning a list with one sum per row. For the grid above it gives `[13, 18, 14]`. You need an outer loop, an accumulator per row, and `append`. This is the hardest thing in today's session and it is worth doing before the lab rather than during it.

PART 6 • 6 MINUTES

The wall: a list is the wrong container

Everything above is real Python. Now here is why it is not enough, and why Friday exists.

6

You have a list of numbers and you want to double every one of them. The obvious thing to try:

this does not do what you want

WRONG

```
[1, 2, 3] * 2
→ [1, 2, 3, 1, 2, 3]      # repeated, not doubled

[1, 2, 3] + 10
→ TypeError: can only concatenate list (not "int") to list
```

This is not a Python defect. A list is a general-purpose container — it can hold numbers, strings, other lists, anything at once — so `*` and `+` mean the only things that make sense for a general container: repeat and join. Arithmetic on every element is not something a list can assume you meant.

So you write a comprehension, and it works. Then you measure it. Here is the same job — square a million numbers — three ways, timed on an ordinary laptop:

255 ms

FOR LOOP + APPEND

200 ms

LIST COMPREHENSION

1.2 ms

NUMPY — FRIDAY

ROUGHLY TWO HUNDRED TIMES FASTER, FOR ONE LINE OF CODE

And a million numbers is nothing. AlexNet did 3.6×10^{17} operations. At the loop's speed that training run would take about three thousand years — which is worse than the 1989 hardware you calculated last week.

The whole of Friday is one idea: stop telling the computer to visit elements one at a time, and start naming the operation you want performed on all of them at once. The difference is not style. It is the difference between this course being possible and not.

LAB 2 • 16 MINUTES IN CLASS

A grid, and the same job written twice

7

Lab 2 is a separate handout — `CSC4315_Lab02_Grids_And_Slices.html`, or the printed copy. Six short tasks: index a grid, slice a list, find where something is, and write one job as a loop and then as a comprehension so you can see they are the same thing.



Same rule as last week

Finish it in the room. Hand up after three minutes of being stuck. And nothing in Lab 2 is a prerequisite for Friday — session 6 starts from a state the whole room shares, whether or not your lab ran.

Before Friday

1. **Finish Lab 2** if the last two tasks did not fit in the room.
2. Write `row_totals` from part 5 without looking, then check it against a grid you make up. Ten minutes. This is the single exercise most likely to make Friday easy.
3. Read your own Lab 1 `total` function again and satisfy yourself that the grid version is the same three steps with one more loop around them. If that sentence does not yet feel true, bring it to me on Friday before class.
4. **Nothing to install.** NumPy came with Anaconda in session 1.



Session 6 – Friday 25 September, 11:30

NumPy I: arrays and shapes. The wall you just hit has a name and a fix. An array is a container that assumes every element is a number of the same kind, which is what lets `a * 2` mean "double every element" and lets it run a hundred times faster than your loop. You will also meet `.shape`, which becomes the first thing you print whenever anything goes wrong for the rest of the course.